

STM32 毕设项目 — 软件需求生成模板 v5

工作流:

1. 你在本模板中填写「项目输入」
2. 整份文档发给 Claude (对话)
3. Claude 输出一份或两份**软件需求文档** (.md 文件)
4. 你把对应的需求文档发给 **Claude Code**, Claude Code 在项目源码中执行开发

你只需要填写下面的「项目输入」部分。框架上下文已内嵌在文档末尾，Claude 会自动参考。

项目输入（每次新题目填写这里）

1. 任务需求【必填】

设计以单片机为核心的最小系统电路、超声波传感器驱动与回波检测电路、LCD 显示电路、按键输入电路及声光报警电路等；同时，使用 C 语言等编程工具开发相应的主控程序、超声波发射/接收时序控制程序、距离计算与液位转换算法、显示程序及报警判断程序，完成从硬件设计制作到软件编程调试的全过程。↵

↵

具体功能及指标要求如下：↵

（1）具备非接触式液位测量功能，系统能通过超声波模块周期性地向液面发射超声波并接收回波，准确测量出传感器到液面的距离。↵

（2）系统测量范围应覆盖常见中小型容器，设定为 5 cm 至 100 cm。在整个量程内，静态液位的测量误差不大于 ± 3 mm。↵

（3）具备液位变化量计算与显示功能，系统能通过记录初始液位高度，并计算与当前测量的差值，实时显示液位的高度变化量，变化量显示分辨率达到 1 mm。↵

（4）具备高低液位报警功能，用户可通过按键自行设定液位上限和下限报警阈值。当液位超过设定范围时，系统能立即通过蜂鸣器和 LED 指示灯发出声光报警信号，报警响应延迟 ≤ 1 秒。↵

（5）使用 LCD 液晶显示屏清晰、直观地显示信息，包括：实时测量距离值、计算得出的当前液位高度、液位变化量、设定的报警阈值以及系统状态（正常/报警）。↵

（6）系统应具备良好的稳定性和抗干扰能力。在室温环境下，对静止液面进行连续测量，其读数波动范围不超过 ± 3 mm。系统应能有效处理因液面轻微波动或环境中突发噪音导致的偶然误测。↵

2. APP 通信变量布局【选填，有 APP 通信则必填】

基于框架的 appcom 模块（remoteVar_TX[16] / remoteVar_RX[16]），规划本项目的变量分配。
不填则由 Claude 根据功能自动分配。

TX (STM32 → APP)

索引	有效宽度	变量含义	数据类型
[0]	4B		
[1]	4B		
[2]	1B		
[3]	1B		
[4]	4B		
[5]	4B		
[6]	1B		
[7]	1B		

RX (APP → STM32)

索引	有效宽度	变量含义	数据类型
[0]	4B		
[1]	4B		
[2]	1B		
[3]	1B		
[4]	4B		
[5]	4B		
[6]	1B		
[7]	1B		

3. APP 需求【选填，有 APP 功能则建议填写】

如果项目包含手机 APP 功能，在此填写 APP 相关信息。留空则 Claude 根据任务需求自动设计。

- 项目名称（显示在 APP 主页顶部）：
- 项目作者（显示在 APP 主页顶部）：

以下为可选补充，留空则由 Claude 自动设计：

- （APP 界面的额外需求，例如：
- 希望 APP 上有哪些特定的显示区域或控制按钮
 - 特定的 UI 风格偏好
 - 其他...

留空则 Claude 根据任务需求自动设计 APP 界面。）

4. 补充说明【选填】

在下位机上使用OLED模块显示所需要的信息，实现功能即可，对于具体的指标所选硬件性能均能够达到。报警功能先不实现，后续我会补充实现。

该液位高度测量装置装在水箱顶部，包含两个超声波测距模块。高精度超声波测距负责测量模块距液面的距离，普通超声波传感器负责测量模块距地面的距离。距地面距离-距液面距离=液位高度。模块上电时，先采集几次普通超声波测距模块的数值获得距地面的距离，从而计算出液面高度。

== 以下为框架上下文（不要修改，Claude 自动参考） ==

<stm32_framework_context>

框架概述

基于 STM32F103C8T6，HAL库 + FreeRTOS CMSIS_V2，分层架构。

开发边界：

- **主要工作：**编写 task 层（应用层）代码，调用 driver 层已有 API
- **允许修改 driver 层的情况：**当某个功能仅靠现有 driver API 无法实现，但通过修改/扩展 driver 层可以实现时，允许修改 driver 层。修改时必须遵守 driver 层的接口规范和命名规范，并在功能实现文档中明确标注修改了哪个 driver 的哪些内容
- **不可修改：**stdlib 层、HAL 层

分层架构与调用规则

task层（userApp）	← 主要工作区，开发应用逻辑
func层（userFunc）	← 可选，跨driver逻辑整合（如appcom已实现）
driver层（userDriver）	← 已完成，现有API不足时允许修改/扩展
stdlib层（userLib）	← 已完成，不可修改
HAL层（Core/）	← CubeMX生成，不可修改

task层可调用：stdlib层、driver层、func层。禁止直接调用 HAL。

文件结构

```
MDK-ARM/
├─ userLib/           # stdlib层（已完成）
├─ userDriver/        # driver层（按项目提供）
├─ userFunc/          # func层（appcom等已实现）
├─ userApp/           # task层（开发工作区）
│   └─ task_system.c/.h  ← 系统任务，不修改
│   └─ task_user1.c/.h  ← 应用任务（主要开发文件）
```

driver 模块自动发现：项目中 `userDriver/` 目录下保留的 `.c/.h` 文件即为本项目全部可用的 driver 模块（项目负责人已删除无关模块）。开发前先浏览 `userDriver/` 目录，逐个阅读每个 driver 的 `.h` 文件中「向上提供」部分，了解可用的结构体和 API，再开始 task 层设计。

driver 层接口规范

每个 driver 模块提供：

1. 公有信息结构体 `xxxInfo`（或 `xxxInfo[CH_COUNT]`），task 层通过它读取状态
2. `DRIVER_XXX_Init()` — 初始化（在 task 的 Init 中调用一次）
3. `DRIVER_XXX_Update()` — 部分 driver 需要周期调用以更新状态
4. 其他功能函数 `DRIVER_XXX_DoSomething(param)`

task 层开发规范

框架有两个 FreeRTOS 任务：

- **task_system**：基础周期 **10ms**，负责 stdlib 状态机维护和 appcom 通信更新。**不需要修改**此文件。
- **task_user1**：基础周期 **2ms**，应用业务逻辑的主要开发文件，通过此文件完成项目功能。

每个任务文件必须包含：

1. 公有信息结构体 `xxxTaskInfo_t` + `extern xxxTaskInfo_t xxxTaskInfo`
2. 初始化函数 `xxxTaskInit(void)`
3. 更新函数 `xxxTaskUpdata(void *argument)` — FreeRTOS 任务入口

标准模板：

```
// task_user1.h
typedef struct {
    uint32_t taskCnt;
    // 任务相关状态数据...
} user1TaskInfo_t;
extern user1TaskInfo_t user1TaskInfo;
void user1TaskInit(void);
void user1TaskUpdata(void *argument);

// task_user1.c
user1TaskInfo_t user1TaskInfo;

void user1TaskInit(void) {
```

```

DRIVER_XXX_Init();
// 初始化任务内部状态
}

void user1TaskUpdata(void *argument) {
    user1TaskInit();
    for(;;) {
        user1TaskInfo.taskCnt++;
        DRIVER_XXX_Update(); // 需要周期更新的driver

        // 通过 taskCnt 取余获得更长的子周期（基础周期 2ms）
        if(user1TaskInfo.taskCnt % 5 == 0) { // 10ms
            // 10ms周期逻辑
        }
        if(user1TaskInfo.taskCnt % 50 == 0) { // 100ms
            // 100ms周期逻辑
        }
        if(user1TaskInfo.taskCnt % 250 == 0) { // 500ms
            // 500ms周期逻辑
        }
        if(user1TaskInfo.taskCnt % 500 == 0) { // 1s
            // 1s周期逻辑
        }

        osDelay(2); // 基础调度周期 2ms
    }
}

```

APP 通信模块 (func_appcom, 已实现)

⚠ 严禁修改 func_appcom.c/h ⚠

func_appcom 中的帧 CMD 编号和帧数量是框架与 ESP32 固件的**固定约定**，修改会导致 BLE 断连。以下参数不可更改：

STM32 TX 帧号（固定）：0x09, 0x0A, 0x0B, 0x0C（共4帧）
 STM32 RX 帧号（固定）：0x13, 0x14, 0x15, 0x16（共4帧）
 帧数量：4 TX + 4 RX（固定）

ESP32 转发时自动做 CMD 偏移，APP 端看到的编号不同，但 STM32 端不需要关心：

- STM32 TX 0x09 → APP RX 0x17 (ESP32 自动偏移)
- STM32 TX 0x0A → APP RX 0x18
- STM32 TX 0x0B → APP RX 0x19
- STM32 TX 0x0C → APP RX 0x1A
- APP TX 0x21 → STM32 RX 0x13 (ESP32 自动偏移)
- APP TX 0x22 → STM32 RX 0x14
- APP TX 0x23 → STM32 RX 0x15
- APP TX 0x24 → STM32 RX 0x16

task 层只通过 `remoteVar_TX[0]~[15]` / `remoteVar_RX[0]~[15]` **读写数据，不需要知道底层帧号。**

通信链路：STM32 → USART1 → ESP32 → BLE → APP（双向）

task 层只操作 `remoteInfo` 结构体，`FUNC_APPCOM_UPDATA()` 由 `task_system` 自动调用。

变量池：`remoteVar_TX[16]` / `remoteVar_RX[16]`，每个元素为 `remoteVar_t`（4字节联合体）：

```
typedef union {
    uint8_t raw[4];
    struct __attribute__((packed)) {
        uint32_t var_uint32;
        int      var_int32;
        float    var_float;
    };
} remoteVar_t;
```

索引与帧槽位映射（TX/RX 各16个）：

- `[0][1][4][5][8][9][12][13]` → 4字节槽位（float/uint32/int32 均可）
- `[2][3][6][7][10][11][14][15]` → 1字节槽位（仅 `var_uint32` 低8位有效）

使用方式：

```
// 发送
remoteInfo.remoteVar_TX[0].var_float = currentTemp;
// 接收
float target = remoteInfo.remoteVar_RX[0].var_float;
uint8_t appKey = remoteInfo.key1;
```

编码约束

约束项	规则
内存分配	禁止 malloc，全部静态分配
注释语言	中文
私有定义	写在 .c 文件中
公有定义	写在 .h 文件中
命名风格	task层：小驼峰（user1TaskInit）；结构体：xxxTaskInfo_t；宏：MODULE_CONSTANT
额外功能	不实现需求未提及的功能

</stm32_framework_context>

<app_framework_context>

APP 框架概述

Android 原生项目，Jetpack Compose + Material Design 3，BLE 通信层已完成。

开发边界：

- **主要工作**：修改 MainActivity.kt 中的 HomeScreen() 函数，实现 UI 展示和控制交互
- **不可修改**：BleProtocol.kt、BleConnectionManager.kt、BleAutoConnector.kt 等通信底层文件

文件结构

```
app/src/main/java/com/example/demo_1/
├─ MainActivity.kt           # 主页面（HomeScreen()）← 主要开发文件
├─ BleScanActivity.kt       # BLE 扫描（不修改）
├─ SecondActivity.kt        # 设置页（不修改）
├─ ThirdActivity.kt         # 调试终端（不修改）
├─ App.kt                   # Application 入口（不修改）
├─ AppBottomNavigation.kt   # 底部导航栏（不修改）
├─ BleProtocol.kt           # 帧解析/构建（不修改）
├─ BleConnectionManager.kt  # GATT 连接/读写（不修改）
├─ BleAutoConnector.kt      # 后台自动连接（不修改）
├─ userConfig.kt            # 全局配置（不修改）
└─ ui/theme/                # 主题（不修改）
```

数据读取方式（RX：STM32 → APP）

APP 接收到的数据通过 BleProtocol.rxFrames 访问，CMD 编号与 STM32 端 TX 帧的对应关系：

STM32 TX CMD	APP RX CMD	说明
0x09	0x17	帧0
0x0A	0x18	帧1
0x0B	0x19	帧2
0x0C	0x1A	帧3

每帧4个槽位：var4b1 (4B)、var4b2 (4B)、var1b1 (1B)、var1b2 (1B)

```
// 读取示例
val frame = BleProtocol.rxFrames[0x17]
val temperature = frame?.var4b1 ?: 0 // 4B_1
val targetTemp  = frame?.var4b2 ?: 0 // 4B_2
val mode        = frame?.var1b1 ?: 0 // 1B_1
val status      = frame?.var1b2 ?: 0 // 1B_2
```

rxFrames 是 mutableStateOf，Compose UI 读取时自动触发重组。

数据发送方式（TX：APP → STM32）

APP TX CMD	STM32 RX CMD	说明
0x21	0x13	帧0

APP TX CMD	STM32 RX CMD	说明
0x22	0x14	帧1
0x23	0x15	帧2
0x24	0x16	帧3

```
// 构建并发送帧
val frame = BleProtocol.buildTxFrame(
    cmd      = 0x21,
    var4_1 = 0,      // 4字节参数1
    var4_2 = 0,      // 4字节参数2
    var1_1 = 0,      // 1字节参数1
    var1_2 = 0       // 1字节参数2
)
val ok = BleConnectionManager.writeCharacteristic(
    serviceUuid      = UserConfig.esp32_service_1_uuid,
    characteristicUuid = UserConfig.esp32_service_1_characteristic_1_uuid,
    value             = frame
)
if (ok) BleConnectionManager.recordOutgoingMessage(
    characteristicUuid = UserConfig.esp32_service_1_characteristic_1_uuid,
    value = frame
)
```

可复用 UI 组件

```
// 分区卡片
HomeSection(title = "系统状态") {
    // 内部放 Row / Text 等
}

// 数据行
ProtoFieldRow(label1 = "温度", value1 = "38.5°C",
               label2 = "目标", value2 = "42°C")
```

APP 端变量与 STM32 端 remoteVar 的索引映射

STM32 端 `remoteVar_Tx[0]~[15]` 对应 APP 端 RX (STM32→APP) :

remoteVar_TX 索引	APP RX CMD	槽位	有效宽度
[0]	0x17	var4b1	4B
[1]	0x17	var4b2	4B
[2]	0x17	var1b1	1B
[3]	0x17	var1b2	1B
[4]	0x18	var4b1	4B

remoteVar_TX 索引	APP RX CMD	槽位	有效宽度
[5]	0x18	var4b2	4B
[6]	0x18	var1b1	1B
[7]	0x18	var1b2	1B
[8]~[11]	0x19	同上规律	—
[12]~[15]	0x1A	同上规律	—

STM32 端 remoteVar_RX[0]~[15] 对应 APP 端 TX (APP→STM32)：

remoteVar_RX 索引	APP TX CMD	槽位	有效宽度
[0]	0x21	var4_1	4B
[1]	0x21	var4_2	4B
[2]	0x21	var1_1	1B
[3]	0x21	var1_2	1B
[4]	0x22	var4_1	4B
[5]	0x22	var4_2	4B
[6]	0x22	var1_1	1B
[7]	0x22	var1_2	1B
[8]~[11]	0x23	同上规律	—
[12]~[15]	0x24	同上规律	—

</app_framework_context>

==== Claude 输出规则（Claude 读到此处后按以下规则输出）====

收到本模板后，Claude 输出 **一份或两份** 软件需求文档（.md 文件），直接发送给对应的 Claude Code 执行：

- **始终输出：**STM32 软件需求文档（给 STM32 项目目录下的 Claude Code）
- **如果任务需求包含 APP 功能：**额外输出 APP 软件需求文档（给 APP 项目目录下的 Claude Code）

文档一：STM32 软件需求文档

一、项目概述

简要说明项目做什么、核心功能有哪些。

二、开发前置步骤

指示 Claude Code：

1. 浏览 MDK-ARM/userDriver/ 目录，阅读每个 driver 的 .h 文件中「向上提供」部分
2. 汇总所有可用的结构体和 API
3. 以此为基础进行后续开发

二（附）、禁止修改的文件清单

必须在 STM32 需求文档中明确列出以下禁止修改的文件，并解释原因：

- **func_appcom.c / func_appcom.h** — 帧 CMD 编号 (TX: 0x09~0x0C, RX: 0x13~0x16) 和帧数量 (4+4) 是框架与 ESP32 固件的固定约定，修改会导致 BLE 断连
- **task_system.c / task_system.h** — 系统任务，框架已实现
- **userLib/ 下所有文件** — stdlib 层，框架已实现
- **Core/ 下所有文件** — HAL 层，CubeMX 生成

同时必须包含以下注意事项：

- 不要整体清零 remoteVar_TX (只写需要的索引，未使用的保持原值)
- 不要在代码中出现 0x17~0x1A、0x21~0x24 等 APP 端帧号 (STM32 端不需要知道这些)

三、task 层设计

1. **文件规划** — 需要哪些 task 文件，各自职责 (默认只用 task_user1)
2. **公有结构体设计** — user1TaskInfo_t 包含哪些字段，逐一说明用途
3. **Init 函数设计** — 调用哪些 DRIVER_XXX_Init(), 初始化哪些状态
4. **Udata 循环设计** — 各功能的执行周期 (基于 2ms 基础周期的 taskCnt 取余)、状态机设计、逻辑流程

四、APP 通信变量布局 (如有)

列出 remoteVar_TX/RX 的完整分配表，说明每个索引对应的变量含义和数据类型。

五、功能详细需求

逐个功能模块详细描述：

- 功能描述
- 实现逻辑 (状态机、算法、判断条件等)
- 涉及的 driver API 调用
- 关键参数和阈值

六、driver 层修改需求（如需要）

如果根据需求分析，现有 driver API 不足以实现某些功能，在此列出：

- 需要修改/新增的 driver 模块
- 修改内容和原因
- 必须遵守 driver 层的接口规范和命名规范

七、输出物要求

指示 Claude Code 完成开发后，必须输出：

1. 代码文件

- task_user1.c / task_user1.h（以及其他需要的 task 文件）
- 如果修改了 driver 层，输出修改后的 driver 文件并用注释标注修改点

2. 功能实现文档（implementation_report.md），包含：

(a) 功能实现状态清单

序号	功能描述	状态	说明
1		✅ / ⚠️ / ❌	

状态分三级：✅ 已实现、⚠️ 部分实现（说明缺什么）、❌ 未实现（说明原因）

(b) driver 层修改记录（如有修改）

文件	修改类型	修改内容	原因
----	------	------	----

(c) 已实现功能的测试步骤

对每个已实现的功能，给出具体的测试方法和预期结果，让开发者可以逐项验证。

文档二：APP 软件需求文档（仅当任务需求包含 APP 功能时输出）

一、项目概述

简要说明 APP 需要实现哪些界面和交互功能。

二、开发边界

指示 Claude Code：

- 只修改 MainActivity.kt 中的 HomeScreen() 函数
- 不修改 BleProtocol.kt、BleConnectionManager.kt 等通信底层文件
- UI 使用 Jetpack Compose + Material Design 3
- 可复用 HomeSection() 和 ProtoFieldRow() 组件

三、通信变量映射表

列出 APP 端需要读/写的所有变量，包含完整的 CMD 和槽位映射：

APP 接收（显示用）：

数据含义	APP RX CMD	槽位	数据类型	对应 STM32 remoteVar_TX 索引
------	------------	----	------	--------------------------

APP 发送（控制用）：

操作含义	APP TX CMD	槽位	数据类型	对应 STM32 remoteVar_RX 索引
------	------------	----	------	--------------------------

并给出具体的读取和发送代码示例。

四、UI 界面设计

强制要求：HomeScreen() 顶部必须包含一个标题区域，显示以下信息（从模板第3节「APP 需求」中获取）：

- 项目名称（大字体居中显示）
- 项目作者（小字体，显示在项目名称下方）

如果用户未填写项目名称/作者，则使用任务需求中的项目名称，作者显示为空。

标题区域之后，用 HomeSection 划分功能区域：

- 每个区域包含哪些显示元素和控制按钮
- 数据显示的格式（如温度保留1位小数、状态用中文文字等）
- 按钮的点击逻辑（调用 buildTxFrame + writeCharacteristic 发送什么数据）

五、交互逻辑

描述 APP 端的业务逻辑：

- 数据刷新方式（Compose 自动重组，无需手动刷新）
- 按钮状态管理（如开关机按钮的文字/颜色切换）
- 异常状态的 UI 表现（故障码展示、颜色变化等）
- 输入校验（如温度设定范围限制）

六、输出物要求

指示 Claude Code 完成开发后，必须输出：

1. 代码文件

- 修改后的 MainActivity.kt （标注修改区域）

2. 功能实现文档（app_implementation_report.md），包含：

- (a) 功能实现状态清单

序号	功能描述	状态	说明
1		✅ / ⚠️ / ❌	

(b) 已实现功能的测试步骤

对每个 APP 功能，给出具体的测试方法和预期结果。

输出约束

- **func_appcom 不可修改（最高优先级）**：STM32 需求文档中必须明确禁止 Claude Code 修改 func_appcom.c/h。帧 CMD 编号（TX: 0x09~0x0C, RX: 0x13~0x16）和帧数量（4TX+4RX）是框架与 ESP32 固件的固定协议，修改会导致 BLE 断连。STM32 需求文档中不得出现 0x17~0x1A、0x21~0x24 等 APP 端帧号
- **不要整体清零 remoteVar_TX**：只写需要的索引，未使用的保持原值。整体清零可能导致未使用帧发送异常数据
- **通信一致性**：Claude 必须先确定 remoteVar_TX/RX 的完整统一布局表，再分别写入 STM32 和 APP 两份文档。两份文档中同一个 remoteVar 索引的变量含义、数据类型、方向必须完全一致
- **APP→STM32 通信方案**：采用直接槽位映射，即每个功能固定占一个 remoteVar_RX 索引（如 RX[0]=设定音量, RX[2]=播放暂停按键），STM32 端直接读取对应索引即可，不使用"指令码+参数"方案
- **STM32 需求文档必须包含验证清单**，至少包括：确认 func_appcom 无改动、确认代码中无 APP 端帧号、确认 remoteVar 索引无越界
- 需求文档中引用框架规范时，直接写明规则（如命名风格、分层约束、API 调用方式），不要让 Claude Code 再去找本模板
- 不要在 STM32 需求文档中生成 CubeMX 配置
- 不要指示 STM32 Claude Code 修改 task_system、stdlib 层、HAL 层
- 不要指示 APP Claude Code 修改 BleProtocol.kt、BleConnectionManager.kt 等通信底层文件
- 两份需求文档应当各自自包含——对应的 Claude Code 拿到它 + 项目源码就能完成开发，不需要其他文档
- APP 需求文档中必须包含完整的 CMD 和槽位映射，不要让 APP 端 Claude Code 去猜 STM32 端的变量布局
- APP 主页面必须包含项目名称和项目作者显示